

Using `peoplelogy/laravel-xero-bridge`

A practical reference for the six Xero flows most applications need: how to call each one, what to pass, what comes back, and what goes wrong.

It covers the package only. Nothing here assumes a particular application, database or domain model — the package deliberately contains no bookkeeping logic, so *when* to raise an invoice and *what* goes on it stay yours to decide.

Every flow below can be run by hand, against a real Xero organisation, from the package's own test console at `/xero/console`. It is there as soon as the package is installed, in every environment except production, and its actions map one-to-one onto the flows in this document — so the quickest way to understand a flow is usually to run it there and read the JSON it returns. Writes are refused unless the connected organisation is a Xero Demo Company or explicitly allow-listed.

Version note. Written against `peoplelogy/laravel-xero-bridge` v1.3.0. Method signatures are read from the package source; request and response samples are reproduced from the package's own docs in this same directory, which are the authority wherever the two disagree. The GUIDs and figures in them are illustrative.

Contents

1. The calling mechanism
 2. Flow 1 — first or create a contact
 3. Flow 2.1 — create a DRAFT invoice
 4. Flow 2.2 — create an invoice that ends up PAID
 5. Flow 3 — TIN, BRN, CC list, then send
 6. Flow 4 — find one invoice
 7. Flow 5 — find a contact
 8. Flow 6 — list invoices
 9. Error handling
 10. Things that differ per organisation
-

The calling mechanism

The namespace

The package autoloader loads PSR-4 from a single root:

```
Peoplelogy\XeroBridge\ → vendor/peoplelogy/laravel-xero-bridge/src/
```

So every class below is `Peoplelogy\XeroBridge\` plus its path — `Filters\InvoiceFilter` is `Peoplelogy\XeroBridge\Filters\InvoiceFilter`, and so on. The service provider is auto-discovered; there is nothing to register in `bootstrap/providers.php`.

Three ways to reach it

1. Import the facade — the usual choice.

```
namespace App\Services\Xero;

use Peoplelogy\XeroBridge\Facades\XeroBridge;

class InvoiceWriter
{
    public function create(): array
    {
        return XeroBridge::invoices()->create([...]);
    }
}
```

2. The global alias, registered by the package as `XeroBridge`. No import, but it lives in the **root** namespace:

```
\XeroBridge::invoices()->create([...]); // note the leading backslash
```

The mistake everybody makes once. Inside a namespaced class, writing `XeroBridge::invoices()` with no `use` and no leading `\` resolves to `App\Services\Xero\XeroBridge` — a class that does not exist — and fatals with *"Class not found"* that names your own namespace, not the package's. Either add the `use` statement or write `\XeroBridge`. In Blade and `routes/*.php` (no namespace) the bare `XeroBridge::` works, which is what makes this inconsistent enough to trip over.

3. Inject the manager — no facade, and trivially mockable in tests:

```
use Peoplelogy\XeroBridge\XeroBridgeManager;

public function __construct(private readonly XeroBridgeManager $xero) {}

// then
$this->xero->invoices()->create([...]);
```

`XeroBridgeManager` is a container singleton, also aliased as `'xero-bridge'`, so `app('xero-bridge')` and `app(XeroBridgeManager::class)` are the same object. The facade is a thin wrapper over exactly this — every method in the tables below works identically through all three.

What you import, and when

Only the facade is needed for ordinary use. The rest you import when a specific job calls for it:

Import	When
<code>Facades\XeroBridge</code>	Always — the entry point
<code>Filters\InvoiceFilter</code>	Building a <code>list()</code> / <code>all()</code> query (flow 6)
<code>Exceptions*</code>	Catching a specific failure (error handling)
<code>Support\InvoiceTransitions</code>	Checking a status change is legal before attempting it
<code>Support\BatchResult</code>	Type-hinting the return of <code>createMany()</code>
<code>Models\XeroConnection</code>	Reading the stored connection row directly
<code>Events*</code>	Writing a listener
<code>Contracts\ConnectionRepository</code>	Replacing how connections are stored

The **resources** — `Resources\Invoices`, `Contacts`, `Payments`, `Settings` — are never imported. You get them from the manager (`XeroBridge::invoices()`), and only type-hint them if you are passing one around.

Everything under `Client\`, `OAuth\`, `Http\`, `Jobs\`, `Repositories\` and `Webhooks\` is internal plumbing. It is reachable, but if you find yourself importing from there, check the facade cannot already do it.

The exceptions, in full

All under `Peoplelogy\XeroBridge\Exceptions\`, all extending `XeroBridgeException`:

```
XeroBridgeException · XeroAuthenticationException · XeroConfigurationException ·
XeroConnectionNotFoundException · XeroReauthorizationRequiredException ·
XeroScopeException · XeroValidationException · XeroRequestException ·
XeroRateLimitException · XeroServiceUnavailableException ·
XeroIdentityUnavailableException · InvalidInvoicePayloadException ·
InvalidInvoiceTransitionException · InvoiceCannotBeVoidedException ·
```

`UnsafeContactPayloadException` · `ConnectionKeyConflictException` · `TenantAlreadyConnectedException`

See error handling for which to catch and why.

Events you can listen to

All under `Peoplelogy\XeroBridge\Events\`, with public readonly properties:

Event	Properties
<code>XeroConnected</code>	<code>XeroConnection \$connection</code> , <code>bool \$wasRepointed</code>
<code>TokenRefreshed</code>	<code>XeroConnection \$connection</code> , <code>?CarbonImmutable \$expiresAt</code>
<code>ConnectionExpired</code>	<code>XeroConnection \$connection</code> , <code>string \$reason</code> , <code>string \$connectUrl</code>
<code>InvoiceCreated</code>	<code>string \$connectionKey</code> , <code>array \$invoice</code> , <code>?string \$idempotencyKey</code>
<code>XeroWebhookReceived</code>	<code>WebhookEvent \$event</code> , <code>WebhookEnvelope \$envelope</code>
<code>XeroWebhookSignatureFailed</code>	<code>?string \$signature</code> , <code>int \$bodyLength</code> , <code>?string \$ip</code>

```
use Peoplelogy\XeroBridge\Events\ConnectionExpired;

Event::listen(ConnectionExpired::class, function (ConnectionExpired $e) {
    Notification::route('mail', config('app.admin_email'))
        ->notify(new XeroNeedsReconnecting($e->reason, $e->connectUrl));
});
```

`ConnectionExpired` is the one worth wiring up: it fires when the refresh token dies, which no amount of retrying fixes — a human has to reconnect.

`XeroWebhookSignatureFailed` fires during Xero's intent-to-receive check by design — it sends deliberately bad signatures to confirm you reject them — so alert on a sustained stream, not on one.

Artisan commands

```
php artisan xero-bridge:install      # publish config + migration, print the .env keys
php artisan xero-bridge:status      # connections, expiry, missing config – no API call
php artisan xero-bridge:refresh-tokens  # run hourly from your scheduler
```

```
// routes/console.php
Schedule::command('xero-bridge:refresh-tokens')->hourly()->withoutOverlapping()->onOneServer();
```

`xero-bridge:status` exits `0` when healthy, `1` on incomplete config, `2` when a connection needs reauthorising — so it can be wired straight into monitoring.

Four resources

```
XeroBridge::contacts()      // Contacts
XeroBridge::invoices()     // Invoices
XeroBridge::payments()     // Payments
XeroBridge::settings()     // Organisation, Accounts, TaxRates, BrandingThemes (read-only)
```

Anything the package has not wrapped is still reachable:

```
XeroBridge::request('GET', 'CreditNotes');           // decoded array
XeroBridge::raw('GET', 'Invoices/{id}');             // the raw Response, for PDFs
XeroBridge::client()->get('Currencies', ['where' => '...']);
```

Which organisation you are talking to

Most applications connect exactly one organisation, in which case you never name it:

```
XeroBridge::invoices()->find('INV-0042');           // the default connection
XeroBridge::connection('acme')->invoices()->find('INV-0042'); // a named one
```

`connection()` returns a **clone**, not `$this`. `XeroBridge::connection('acme')->invoices()` is scoped to that expression alone and cannot leak into the next call in the same request. The default connection name comes from `XERO_DEFAULT_CONNECTION` (`default`).

What you get back

Every method returns **plain PHP arrays, already unwrapped** from Xero's envelope. Xero answers `POST /Invoices` with `{"Invoices": [ ... ]}`; `create()` hands you the inner element.

Shape	Methods
<code>array</code> — the record	<code>create()</code> , <code>update()</code> , <code>authorise()</code> , <code>organisation()</code>
<code>?array</code> — <code>null</code> when absent, no exception	<code>find()</code> , <code>findByEmail()</code>
<code>list<array></code> — possibly empty	<code>list()</code> , <code>accounts()</code> , <code>taxRates()</code>
<code>void</code>	<code>email()</code> — Xero answers 204 with no body
<code>string</code>	<code>pdf()</code> (raw bytes), <code>onlineUrl()</code>

What it does for you

Token refresh (access tokens last 30 minutes, refresh tokens rotate and are refreshed under a per-connection lock), retries with backoff, `Retry-After` handling, `Idempotency-Key` on writes, and typed exceptions. You do not write any of that.

Dates

Send plain `Y-m-d` strings. Xero **returns** `/Date(1772409600000+0000)/`, which is not JSON-parseable as a date — read `DateString` / `DueDateString` instead, or convert the milliseconds yourself. A date read back from Xero and re-posted unchanged will be rejected.

Flow 1 — first or create a contact

```
$contact = XeroBridge::contacts()->firstOrCreate(
    ['EmailAddress' => 'finance@acme.com'],    // exactly ONE lookup key
    [                                           // merged in only when creating
        'Name' => 'Acme Sdn Bhd',
        'FirstName' => 'Siti',
        'LastName' => 'Rahman',
        'CompanyNumber' => '202401012345',    // BRN
        'TaxNumber' => 'C12345678901',       // TIN
    ],
);
```

Parameters

Argument	Type	Notes
<code>\$lookup</code>	<code>array<string,string></code>	Exactly one key, either <code>EmailAddress</code> or <code>Name</code> . Anything else throws <code>InvalidArgumentException</code> .
<code>\$attributes</code>	<code>array<string,mixed></code>	Applied only on create . An existing contact comes back untouched.

On the wire

A lookup first (`GET /Contacts?where=EmailAddress=="finance@acme.com"`), then, only if nothing matched, a create:

```
PUT /api.xro/2.0/Contacts
{
  "Contacts": [
    {
      "EmailAddress": "finance@acme.com",
      "Name": "Acme Sdn Bhd",
      "CompanyNumber": "202401012345",
      "TaxNumber": "C12345678901"
    }
  ]
}
```

Response

```
{
  "ContactID": "8138a266-fb42-49b2-a104-014b7045753d",
  "ContactStatus": "ACTIVE",
  "Name": "Acme Sdn Bhd",
  "EmailAddress": "finance@acme.com",
  "CompanyNumber": "202401012345",
  "TaxNumber": "C12345678901",
  "ContactPersons": [],
  "IsCustomer": true,
  "UpdatedDateUTC": "/Date(1790553726193+0000)/",
  "HasValidationErrors": false
}
```

Persist `ContactID`. It is the only stable way to reference a contact.

Gotchas

- **It is a `PUT`, not a `POST`, on purpose.** `POST /Contacts` is an implicit upsert: matched on a non-unique name, it silently overwrites a real customer record. Xero's own guidance is to reference contacts by `ContactID`.
- **Race-safe.** If another process creates the contact between the lookup and the create, the loser gets a clean conflict and re-reads, rather than clobbering the winner.
- **Never use it to update.** An existing contact ignores `$attributes` entirely. To change a contact, call `contacts()->update($contactId, [...])`.
- `CompanyNumber` and `TaxNumber` are capped at 50 characters by Xero.

Flow 2.1 — create a DRAFT invoice

```
$invoice = XeroBridge::invoices()->create([
  'Type' => 'ACCREC', // REQUIRED
  'Status' => 'DRAFT',
  'Contact' => ['ContactID' => '1111...'], // ContactID ALONE
  'Date' => '2026-03-02',
  'DueDate' => '2026-04-01',
  'Reference' => 'CCF-2026-0042',
  'LineItems' => [[
    'Description' => 'Leadership training, 2 days',
    'Quantity' => 1,
    'UnitAmount' => 4800,
    'AccountCode' => '200', // omit to use the connection default
    'TaxType' => 'OUTPUT', // omit to use the connection default
  ]],
]);
```

Parameters

Field	Required	Notes
Type	yes	ACCREC (sales) or ACCPAY (bill). Missing throws InvalidInvoicePayloadException before anything is sent.
Contact	yes	['ContactID' => ...] — see the gotcha below.
LineItems	yes	At least one. Description is the minimum.
Status	no	Omitted creates a DRAFT.
Date, DueDate, Reference	no	Plain Y-m-d.
AccountCode, TaxType, CurrencyCode, BrandingThemeID	no	Filled from the connection defaults only where you left them out .

On the wire

POST /api.xro/2.0/Invoices, with an Idempotency-Key header:

```
{
  "Invoices": [
    {
      "Type": "ACCREC",
      "Contact": { "ContactID": "11111111-1111-1111-1111-111111111111" },
      "Date": "2026-03-02",
      "DueDate": "2026-04-01",
      "Reference": "CCF-2026-0042",
      "Status": "DRAFT",
      "LineItems": [
        {
          "Description": "Leadership training, 2 days",
          "Quantity": 1,
          "UnitAmount": 4800,
          "TaxType": "OUTPUT",
          "AccountCode": "200"
        }
      ],
      "CurrencyCode": "MYR"
    }
  ]
}
```

Response

The method returns the inner invoice, not the envelope:

```
{
  "Type": "ACCREC",
  "InvoiceID": "22222222-2222-2222-2222-222222222222",
  "InvoiceNumber": "INV-0042",
  "Reference": "CCF-2026-0042",
  "Status": "DRAFT",
  "AmountDue": 5184.00,
  "AmountPaid": 0.00,
  "SubTotal": 4800.00,
  "TotalTax": 384.00,
  "Total": 5184.00,
  "CurrencyCode": "MYR",
  "DateString": "2026-03-02T00:00:00",
  "DueDateString": "2026-04-01T00:00:00",
  "SentToContact": false,
  "Contact": { "ContactID": "1111...", "Name": "Acme Sdn Bhd" },
  "LineItems": [
    {
      "LineItemID": "33333333-3333-3333-3333-333333333333",
      "Description": "Leadership training, 2 days",
      "UnitAmount": 4800.00,
      "TaxType": "OUTPUT",
      "TaxAmount": 384.00,
      "LineAmount": 4800.00,
      "AccountCode": "200",
      "Quantity": 1.0000
    }
  ]
}
```

Persist `InvoiceID` and `InvoiceNumber`. Note that **tax is computed server-side** — you sent 4800, Xero returned a `Total` of 5184.

Gotchas

- `Contact` **must carry** `ContactID` **and nothing else**. With `ContactID` plus any other field, Xero applies those fields to the *contact record* and **deletes any** `ContactPersons` **you left out**. The package refuses this with `UnsafeContactPayloadException` rather than letting it happen; `withContactMutation()` opts in if you genuinely mean it. It is irreversible and nobody notices for weeks.
- A `DRAFT` can be deleted outright. Once `AUTHORISED` it can only be voided, and it stays visible in Xero forever.
- An unknown `AccountCode` is a 400 → `XeroValidationException`, with Xero's own message in `validationErrors()`.
- The `Idempotency-Key` is retained by Xero for **six minutes**. It protects an immediate network retry; it does **not** deduplicate a queued job retried an hour later. Persist `InvoiceID` for that.

Flow 2.2 — create an invoice that ends up PAID

PAID is not a status you can set

Xero rejects `"Status": "PAID"` on create. `PAID` is what Xero moves an invoice to once payments cover it in full. There is no single call for this.

Three steps, in this order:

```
// 1. AUTHORISED, not DRAFT – a payment cannot attach to a draft
$invoice = XeroBridge::invoices()->create([
    'Type' => 'ACCREC',
    'Status' => 'AUTHORISED',
    'Contact' => ['ContactID' => $contactId],
    'LineItems' => [[ 'Description' => '...', 'Quantity' => 1, 'UnitAmount' => 4800 ]],
]);

// 2. pay Xero's own AmountDue, never the figure you sent
$payment = XeroBridge::payments()->createForInvoice(
    $invoice['InvoiceID'],
    (float) $invoice['AmountDue'], // 5184.00, not 4800
    '090', // a BANK / payments-enabled account code
    '2026-03-02', // optional, defaults to today
);

// 3. read it back – do not assume
$settled = XeroBridge::invoices()->find($invoice['InvoiceID']);
// $settled['Status'] === 'PAID'
```

`createForInvoice()` parameters

Argument	Type	Notes
<code>\$invoiceId</code>	<code>string</code>	Must be AUTHORISED .
<code>\$amount</code>	<code>float int string</code>	Use the invoice's <code>AmountDue</code> .
<code>\$accountCode</code>	<code>?string</code>	Type <code>BANK</code> , or with "enable payments to this account" on. Discover with <code>settings()->paymentAccounts()</code> . Throws <code>InvalidArgumentException</code> if null and no connection default.
<code>\$date</code>	<code>DateTimeInterface string null</code>	Defaults to today.
<code>\$extra</code>	<code>array</code>	Merged into the payload.

On the wire

```
POST /api.xro/2.0/Payments
{
  "Payments": [
    {
      "Invoice": { "InvoiceID": "c7c37b83-ac95-45ea-88ba-8ad83a5f22fe" },
      "Account": { "Code": "090" },
      "Amount": 5184,
      "Date": "2026-03-02"
    }
  ]
}
```

Gotchas

- Pay `AmountDue`, not your own total. Tax and rounding are applied server-side. Paying the pre-tax figure leaves the invoice `AUTHORISED` with a balance, not `PAID`.
- A `PAID` invoice can never be changed. `InvoiceTransitions` lists no legal transition out of it. To correct one, reverse the payment first with `payments()->delete($paymentId)`, then void.
- Overpaying is rejected by Xero — check `AmountDue` before applying.
- A successful 200 can still carry a `Warnings` array (a `CurrencyRate` warning, typically). The package logs it; check it yourself if a wrong foreign-currency amount would matter.

Flow 3 — TIN, BRN, CC list, then send

Three facts shape this flow, and all three surprise people.

`POST /Invoices/{id}/Email` takes an empty body

No `to`, no `cc`, no `bcc`, no `subject`, no `body`, no `replyTo`. Xero's own OpenAPI spec types the request body as `RequestEmpty`. The package sends `[]` on the wire and has a test asserting it stays empty.

1. **TIN and BRN live on the contact, not the invoice.** In a Malaysian organisation the TIN goes in `TaxNumber` (the field is relabelled per region — ABN in Australia, VAT Number in the UK); the BRN is `CompanyNumber`. Both max 50 characters.
2. **ContactPersons with IncludeInEmails is the only CC mechanism.** Maximum five. The list you send **replaces** the stored one wholesale, so build it in full every time. It is **per contact, not per send** — everyone on it is copied on every future invoice for that customer until you change it back.
3. **Xero only emails a SUBMITTED, AUTHORISED or PAID invoice**, so a DRAFT must be approved first.

Validate the TIN before you store it

From v1.3.0 the package can check the buyer's TIN against LHDN Malaysia's MyInvois API before it is written onto the contact, rather than trusting whatever was typed:

```
use Peoplelogy\XeroBridge\MyInvois\Facades\MyInvois;
use Peoplelogy\XeroBridge\MyInvois\IdType;

if (! MyInvois::validate($tin, IdType::BRN, $brn)) {
    // HASiL has no record of this pair. Ask the buyer to confirm both values
    // rather than writing them to the contact and on to the invoice.
}
```

Three things about it matter here:

- **It needs BOTH values.** Since 1 August 2026 LHDN validates the TIN and the registration number *as a pair*, so a valid TIN with a stale or mistyped BRN answers exactly like a fabricated one. That is a reason to collect the BRN carefully, not a reason to distrust the check.
- **A negative answer is not an error.** LHDN returns HTTP 404 for "no such pair", which is the answer you asked for, so the method returns `false`. Only a genuine failure — a malformed request, a credential problem, a rate limit, an outage — throws.
- **A positive answer is narrower than it looks.** It proves the pair exists in HASiL's records. It does *not* prove the pair belongs to the customer you are invoicing: the endpoint returns no name and no address at all.

The module is **off by default** and needs `MYINVOIS_ENABLED=true` with credentials from the MyInvois portal. Full detail in 08-myinvois-tin-validation.md.

```
// (a) the buyer's details, onto the CONTACT
XeroBridge::contacts()->update($contactId, [
    'CompanyNumber' => '202401012345', // BRN
    'TaxNumber' => 'C12345678901', // TIN
    'ContactPersons' => [
        ['FirstName' => 'Siti', 'LastName' => 'Rahman',
        'EmailAddress' => 'ap@acme.com', 'IncludeInEmails' => true],
        ['FirstName' => 'Finance', 'LastName' => 'Team',
        'EmailAddress' => 'finance@acme.com', 'IncludeInEmails' => true],
    ],
]);

// (b) Xero refuses to email a DRAFT
XeroBridge::invoices()->authorise($invoiceId);

// (c) send. 204 No Content, so this returns void.
XeroBridge::invoices()->email($invoiceId);
```

On the wire

`POST /Contacts/{ContactID}`:

```
{
  "Contacts": [
    {
      "CompanyNumber": "202401012345",
      "TaxNumber": "C12345678901",
      "ContactPersons": [
        { "FirstName": "Siti", "LastName": "Rahman",
          "EmailAddress": "ap@acme.com", "IncludeInEmails": true }
      ]
    }
  ]
}
```

then `POST /Invoices/{InvoiceID}` with nothing but a status:

```
{ "Invoices": [ { "Status": "AUTHORISED" } ] }
```

then `POST /Invoices/{InvoiceID}/Email` with an empty body.

Who actually receives it

The contact's own `EmailAddress`, plus every `ContactPerson` whose `IncludeInEmails` is `true`. That is the entire mechanism.

Gotchas

- **You cannot copy someone on one invoice only.** Changing who is copied means changing the contact, and that applies to every subsequent invoice for that customer.
- **The template and sender are not yours.** Subject and body come from the organisation's email template, configured in Xero's UI. The sender is the user who authorised the app connection.
- **You cannot read back what was sent.** There is no API for the rendered email. If you need an audit trail, log the `ContactPersons` you set and the moment you called `email()`.
- If none of that fits, send it yourself: `invoices()->pdf($id)` for the bytes or `invoices()->onlineUrl($id)` for the public link, build your own Mailable, then call `invoices()->markAsSent($id)` so Xero's UI still shows it as sent.

Flow 4 — find one invoice

```
$invoice = XeroBridge::invoices()->find('INV-0042'); // or the InvoiceID GUID

if ($invoice === null) {
    // no such invoice – this is NOT an exception
}
```

Argument	Notes
<code>\$idOrNumber</code>	An <code>InvoiceID</code> GUID or a human <code>InvoiceNumber</code> such as <code>INV-0042</code> .

GET `/api.xro/2.0/Invoices/{id}`. Returns the same shape as 2.1, including full `LineItems` and any `Payments` applied.

Gotcha: returns `null` for "not found" rather than throwing, so `??` and `is_null()` work naturally — but an unchecked `$invoice['Status']` will fatal on a miss.

Flow 5 — find a contact

5.1 By email — `contacts.find_by_email`

```
$contact = XeroBridge::contacts()->findByEmail('finance@acme.com'); // ?array
```

GET `/Contacts?where=EmailAddress=="finance@acme.com"`. Returns the **first** match or `null`.

Gotcha: Xero allows more than one contact per email address. Treat this as *a* match, not *the* match. `findAllByEmail()` returns every one if that distinction matters.

5.2 By ContactID — `contacts.find`

```
$contact = XeroBridge::contacts()->find('8138a266-fb42-49b2-a104-014b7045753d'); // ?array
```

GET `/Contacts/{ContactID}`. The only unique way to reference a contact — Xero explicitly warns that contact names are not unique and recommends `ContactID` everywhere.

Both return the shape shown under flow 1.

Flow 6 — list invoices

```
use Peoplelogy\XeroBridge\Filters\InvoiceFilter;

$invoices = XeroBridge::invoices()->list(
    InvoiceFilter::make()
        ->statuses(['AUTHORISED', 'PAID'])
        ->type('ACCREC')
        ->dateBetween('2026-01-01', '2026-03-31')
        ->orderBy('Date', 'DESC')
        ->page(1, 25)
);
```

Returns `list<array>` — one page. For everything matching, `all()` returns a `LazyCollection` and walks Xero's pagination for you:

```
XeroBridge::invoices()->all($filter)->each(fn (array $invoice) => /* ... */);
```

Filter methods

Method	Notes
<code>statuses(string array)</code>	<code>DRAFT</code> , <code>SUBMITTED</code> , <code>AUTHORISED</code> , <code>PAID</code> , <code>VOIDED</code> , <code>DELETED</code>
<code>type(string)</code>	<code>ACCREC</code> or <code>ACCPAY</code>
<code>contactIds()</code> , <code>contactName()</code>	
<code>invoiceNumbers()</code> , <code>ids()</code> , <code>reference()</code>	
<code>dateBetween(\$from, \$to)</code>	Either side may be null
<code>modifiedSince(\$when)</code>	Sent as the <code>If-Modified-Since</code> header, not a query param
<code>orderBy(\$field, \$dir)</code>	
<code>page(\$page, \$pageSize)</code> , <code>pageSize()</code>	
<code>summaryOnly()</code>	Lighter rows, no line items
<code>createdByMyApp()</code>	Only invoices this app created
<code>whereRaw()</code> , <code>whereEquals()</code> , <code>whereGuid()</code>	Escape hatches

Gotchas

- **Always page.** An unbounded `/Invoices` on a live organisation is the quickest way to spend the daily rate limit. `all()` pages at 100 unless told otherwise.
- **Xero caps a page at 1000 and silently clamps anything larger**, so `pageSize()` throws outside 1–1000 rather than let a typo'd `5000` look like it worked.

- `list()` gives one page and ignores the rest. `all()` is the one that walks.
- `summaryOnly()` omits `LineItems` — don't reach into them on a summary row.
- `modifiedSince()` is a header, so it does not appear in the query string when debugging.

Error handling

Every failure is a typed exception extending `Peoplelogy\XeroBridge\Exceptions\XeroBridgeException`. Catch the specific one when you can act on it differently; catch the base class as a backstop.

```
use Peoplelogy\XeroBridge\Exceptions\XeroBridgeException;
use Peoplelogy\XeroBridge\Exceptions\XeroRateLimitException;
use Peoplelogy\XeroBridge\Exceptions\XeroValidationException;

try {
    $invoice = XeroBridge::invoices()->create($payload);
} catch (XeroValidationException $e) {
    // Xero rejected the payload. $e->validationErrors() holds its own messages.
    report($e);
} catch (XeroRateLimitException $e) {
    // $e->retryAfter() is seconds, or null on a concurrency limit.
    $this->release($e->retryAfter() ?? 60);
} catch (XeroBridgeException $e) {
    report($e);
}
```

What to catch, and what it means

Exception	Cause	What to do
<code>XeroConfigurationException</code>	A <code>XERO_*</code> key is missing or wrong (bad redirect URI, no <code>offline_access</code>)	Fix <code>.env</code> , <code>php artisan config:clear</code> . Never retried.
<code>XeroConnectionNotFoundException</code>	Nothing connected under that key	Send an admin to <code>/xero/connect</code>
<code>XeroReauthorizationRequiredException</code>	The refresh token is dead (terminal <code>invalid_grant</code>)	Reconnect. No code change helps.
<code>XeroScopeException</code>	The connection was authorised without a scope this call needs	Add it to <code>XERO_SCOPES</code> and reconnect — an existing connection never gains scopes
<code>XeroValidationException</code>	400 from Xero: unknown <code>AccountCode</code> , missing contact, bad <code>TaxType</code>	Fix the payload. <code>validationErrors()</code> names the element.
<code>UnsafeContactPayloadException</code>	An invoice <code>Contact</code> block carries <code>ContactID</code> plus other fields	Send <code>ContactID</code> alone
<code>InvalidInvoicePayloadException</code>	<code>Type</code> missing on create	Nothing was sent
<code>InvalidInvoiceTransitionException</code>	e.g. voiding a DRAFT	Checked locally against <code>InvoiceTransitions</code> , so the message names the current status
<code>XeroRateLimitException</code>	429	<code>retryAfter()</code> on minute/daily limits; null on a concurrency limit — back off yourself
<code>XeroServiceUnavailableException</code>	Organisation offline, or Xero is down	Retry in a few minutes
<code>XeroRequestException</code>	Anything else non-2xx	Inspect <code>statusCode()</code> and <code>response()</code>

Useful accessors on any `XeroBridgeException`

```

$e->statusCode();      // ?int
$e->connectionKey();    // ?string
$e->validationErrors(); // array — Xero's own messages
$e->xeroType();         // ?string
$e->xeroErrorNumber();  // ?int
$e->retryAfter();       // ?int seconds
$e->response();         // ?Illuminate\Http\Client\Response
$e->context();          // array

```

In a queued job

`retryAfter()` exists so a worker can `release()` itself instead of sleeping. A daily rate limit can carry a `Retry-After` measured in hours — never sleep inline on one.

Things that differ per organisation

Nothing below should ever be hardcoded. These differ between Xero organisations, so a value that works in the sandbox will be rejected in production.

What	How to discover it
Account codes	<code>XeroBridge::settings()->accounts()</code> — a <code>200</code> sales account in one org is <code>4000</code> in another
Payment accounts	<code>XeroBridge::settings()->paymentAccounts()</code> — type <code>BANK</code> or payments-enabled only
Tax types	<code>XeroBridge::settings()->taxRates()</code> — the <code>TaxType</code> code, not the label
Branding themes	<code>XeroBridge::settings()->brandingThemes()</code>
Base currency, region, <code>IsDemoCompany</code>	<code>XeroBridge::settings()->organisation()</code>

This is also why three of the four per-connection defaults ship **no value at all**. Only `XERO_CURRENCY` has one, and only because a base currency is a property of the organisation rather than of a line:

Setting	Ships a default?
<code>XERO_ACCOUNT_CODE</code>	no — the <code>200</code> fallback was removed in v1.1.0
<code>XERO_TAX_TYPE</code>	no
<code>XERO_BRANDING_THEME_ID</code>	no
<code>XERO_CURRENCY</code>	yes, <code>MYR</code>

A wrong-but-present account code is worse than a missing one: Xero accepts the invoice and posts it to whatever account that code happens to be in that organisation, and nobody notices until Finance reconciles. With no default, Xero rejects the invoice and names the problem.

`XERO_TAX_TYPE` is unset for a second reason. Tax is frequently **per line**, not per invoice — under Malaysian SST, for instance, training is 8% while education and rental are 6%, so a single invoice can legitimately carry two rates. A connection-wide default would quietly stamp the wrong one on a venue recharge. Set it only if every line of every invoice on that connection genuinely carries the same rate.

None of these are cached by the package. The right cache key, TTL and invalidation depend entirely on the consuming application, so caching belongs there.

The same caution applies to a MyInvois TIN check. A match tells you the TIN and registration number exist together in HASiL's records and nothing further — no name is returned, so it is not evidence of who you are dealing with. And because the pair has only been validated together since 1 August 2026, any verdict stored before that date was answering a weaker question and is worth re-running.

Related

- The numbered guides in this directory — [01-getting-started](#), [02-invoices](#), [03-contacts-payments-settings](#), [04-webhooks-and-events](#), [05-commands-and-errors](#), [06-recipes](#), [07-test-console](#). Deeper than this page on every topic, and the authority where the two disagree.
- The test console at [/xero/console](#) — runs every flow below by hand and shows the request timings, the rate limit Xero reported and the full response. See [07-test-console](#) for the write guard and for how it is switched on and off.
- `php artisan xero-bridge:status` — connections, token expiry and missing configuration, without calling Xero
- [08-myinvois-tin-validation](#) in the package docs — the optional LHDN MyInvois taxpayer TIN validator: turning it on, the 404-is-an-answer contract, caching and troubleshooting
- Xero Accounting API reference — for the fields this package passes through untouched